

Guide Git & GitHub

1. Qu'est-ce que Git ?

Git est un système de gestion de versions (*Version Control System*) qui permet aux développeurs de :

- Sauvegarder les différentes versions d'un projet
- Suivre les modifications du code
- Travailler en équipe
- Restaurer une ancienne version en cas de problème

Exemple :

Pendant le développement d'une application :

Version 1 → Création de la page d'accueil

Version 2 → Ajout du système de connexion

Version 3 → Correction des erreurs

Git garde l'historique de toutes ces modifications.

2. Qu'est-ce que GitHub ?

GitHub est une plateforme en ligne qui permet d'héberger des projets Git.

GitHub permet de :

- Stocker les projets sur Internet
- Partager du code avec d'autres développeurs
- Collaborer en équipe
- Gérer les versions d'un projet

Relation entre Git et GitHub :

Git = Outil installé sur votre ordinateur

GitHub = Serveur en ligne pour stocker vos projets Git

Architecture :

Ordinateur personnel

|
| git push
↓
Repository GitHub

3. Installation de Git

Télécharger Git :

<https://git-scm.com/downloads>

Vérifier l'installation :

git --version

Exemple :

git version 2.50.0

4. Configuration de Git (première utilisation)

Avant de travailler avec Git, il faut configurer votre identité.

Définir le nom :

git config --global user.name "Votre Nom"

Exemple :

git config --global user.name "Ibrahim Jlidi"

Définir l'adresse email :

git config --global user.email "email@gmail.com"

Vérifier la configuration :

git config --list

5. Créer un compte GitHub

Accéder au site :

<https://github.com>

Créer un compte puis :

1. Cliquer sur **New Repository**
2. Donner un nom au projet
3. Choisir Public ou Private
4. Cliquer sur **Create repository**

Exemple :

Nom du repository :

Formation-Mern

6. Créer un projet Git local

Créer un dossier :

mkdir mon-projet

Entrer dans le dossier :

cd mon-projet

Initialiser Git :

git init

Résultat :

Initialized empty Git repository

Git crée un dossier caché :

mon-projet

```
|
|— .git
|
|— index.html
|— style.css
```

Le dossier `.git` contient l'historique du projet.

7. Le fonctionnement de Git

Git utilise trois zones principales :

Working Directory

|
| git add



Staging Area

|
| git commit



Repository local

|
| git push



GitHub

8. Vérifier l'état du projet

Commande :

git status

Exemple :

Untracked files:

index.html
style.css

Cela signifie que Git détecte de nouveaux fichiers.

9. Ajouter des fichiers à Git

Ajouter un fichier :

git add index.html

Ajouter tous les fichiers :

git add .

Le fichier est maintenant prêt à être sauvegardé.

10. Créer un commit

Un **commit** représente une sauvegarde d'une version du projet.

Commande :

git commit -m "Message du commit"

Exemple :

git commit -m "Création de la page d'accueil"

Bonnes pratiques :

✓ Bons messages :

Ajouter la page login

Corriger le bug du formulaire

Créer le modèle utilisateur

✗ Mauvais messages :

test

modification

update

11. Connecter Git avec GitHub

Copier l'URL du repository GitHub.

Exemple :

<https://github.com/user/projet.git>

Ajouter le lien :

git remote add origin https://github.com/user/projet.git

Vérifier :

git remote -v

12. Envoyer le projet vers GitHub (Push)

Changer le nom de la branche principale :

git branch -M main

Envoyer le projet :

git push -u origin main

Maintenant le projet est disponible sur GitHub.

13. Télécharger un projet GitHub (Clone)

Pour récupérer un projet existant :

git clone URL_DU_PROJET

Exemple :

git clone https://github.com/user/projet.git

14. Mettre à jour un projet

Récupérer les dernières modifications :

git pull

Cycle de travail classique :

Modifier le code

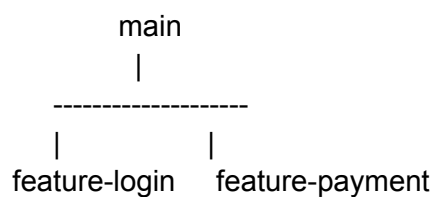
|
↓
git add .
|

↓
git commit -m "message"
↓
git push

15. Les branches Git

Une branche permet de travailler sur une fonctionnalité sans modifier la version principale.

Exemple :



Créer une branche :

git branch login

Changer de branche :

git checkout login

Nouvelle commande recommandée :

git switch login

16. Fusionner une branche (Merge)

Après terminer une fonctionnalité :

Retourner sur main :

git switch main

Fusionner :

git merge login

17. Travail en équipe avec GitHub

Exemple :

Étudiant A

Clone le projet :

git clone URL

Crée une branche :

git switch -c ajout-navbar

Travaille puis :

git add .

git commit -m "Ajout navbar"

Envoie sa branche :

git push origin ajout-navbar

Ensuite il crée une **Pull Request** sur GitHub.

18. Fichier .gitignore

Certains fichiers ne doivent pas être envoyés sur GitHub.

Créer un fichier :

.gitignore

Exemple pour un projet Node.js :

node_modules/

.env

dist/

Exemple React :

node_modules

.env

build

19. Commandes Git importantes

Commande	Description
<code>git init</code>	Créer un dépôt Git
<code>git status</code>	Voir les modifications
<code>git add .</code>	Ajouter les fichiers
<code>git commit -m "</code>	Sauvegarder une version
<code>git log</code>	Voir l'historique
<code>git push</code>	Envoyer vers GitHub
<code>git pull</code>	Récupérer les changements
<code>git clone</code>	Copier un projet
<code>git branch</code>	Voir les branches
<code>git switch</code>	Changer de branche
<code>git merge</code>	Fusionner des branches